

```

import os
import json
import logging
import asyncio
from datetime import datetime, timezone

from dotenv import load_dotenv
from openai import AsyncOpenAI

from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import (
    Application,
    CommandHandler,
    MessageHandler,
    CallbackQueryHandler,
    ContextTypes,
    filters,
)

# =====
# CONFIG
# =====

load_dotenv()

BOT_TOKEN = os.getenv("BOT_TOKEN")
OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")

# Where the JSON "database" lives on disk.
# On Railway, mount a persistent volume at this path (e.g. /data)
# so data survives redeploy. Defaults to a local file otherwise.
DATA_FILE = os.getenv("DATA_FILE", "properties.json")

ADMIN_IDS = {
    int(x.strip())
    for x in os.getenv("ADMIN_IDS", "").split(",")
    if x.strip().isdigit()
}

if not BOT_TOKEN:
    raise RuntimeError("BOT_TOKEN غير موجود")

if not OPENAI_API_KEY:

```

```

        raise RuntimeError("OPENAI_API_KEY غير موجود")

# =====
# LOGGING
# =====

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s",
)

logger = logging.getLogger("alshahba")

# =====
# JSON STORAGE
# =====
#
# Replaces the previous SQLAlchemy/Postgres setup with a simple
# JSON file on disk. A lock protects against concurrent writes
# since Telegram updates can arrive while a write is in progress.

_storage_lock = asyncio.Lock()

def _load_properties_sync():
    if not os.path.exists(DATA_FILE):
        return []

    try:
        with open(DATA_FILE, "r", encoding="utf-8") as f:
            content = f.read().strip()
            if not content:
                return []
            return json.loads(content)
    except (json.JSONDecodeError, OSError) as e:
        logger.error("Failed to read %s: %s", DATA_FILE, e)
        return []

def _save_properties_sync(properties):
    tmp_path = DATA_FILE + ".tmp"

    with open(tmp_path, "w", encoding="utf-8") as f:
        json.dump(properties, f, ensure_ascii=False, indent=2)

```

```

os.replace(tmp_path, DATA_FILE)

async def load_properties():
    async with _storage_lock:
        return _load_properties_sync()

async def save_properties(properties):
    async with _storage_lock:
        _save_properties_sync(properties)

async def add_property(record):
    """Append a new property record and return it (with id/code set)."""

    async with _storage_lock:
        properties = _load_properties_sync()

        next_id = (max((p.get("id", 0) for p in properties), default=0)) + 1
        record["id"] = next_id
        record["property_code"] = f"SH-{next_id:04d}"

        now = datetime.now(timezone.utc).isoformat()
        record["created_at"] = now
        record["updated_at"] = now

        properties.append(record)

        _save_properties_sync(properties)

    return record

async def init_storage():
    if not os.path.exists(DATA_FILE):
        _save_properties_sync([])

    logger.info("JSON storage ready at %s", DATA_FILE)

# =====
# OPENAI
# =====

client = AsyncOpenAI(
    api_key=OPENAI_API_KEY

```

)

```
# =====  
# PROPERTY EXTRACTION  
# =====
```

```
EXTRACTION_PROMPT = ""
```

. أنت نظام استخراج بيانات عقارات لمكتب الشهباء العقاري.

. مهمتك استخراج المعلومات الموجودة فعلياً في النص أو الصور.

. ممنوع التخمين.

: إذا لم تكن المعلومة موجودة أو واضحة
null. اجعل قيمتها.

: إذا كان هناك تعارض بين النص والصورة
conflicts. ضع التعارض داخل

. فقط JSON أخرج.

: المفاتيح المطلوبة

```
{  
  "operation": null,  
  "property_type": null,  
  "city": null,  
  "district": null,  
  "neighborhood": null,  
  "address": null,  
  "area_m2": null,  
  "rooms": null,  
  "bathrooms": null,  
  "floor": null,  
  "total_floors": null,  
  "building_age": null,  
  "furnished": null,  
  "price": null,  
  "currency": null,  
  "description": null,  
  "owner_name": null,  
  "owner_phone": null,  
  "conflicts": [],  
  "missing_fields": []  
}
```

.السعر يجب أن يكون رقماً فقط
.المساحة يجب أن تكون رقماً بالمتر المربع
.عدد الغرف رقماً
.لا تخرع أي معلومة
"""

```
async def extract_property(text, image_urls=None):

    content = [
        {
            "type": "input_text",
            "text": EXTRACTION_PROMPT + "\n\nالنص:\n" + (text or "")
        }
    ]

    if image_urls:
        for url in image_urls:
            content.append(
                {
                    "type": "input_image",
                    "image_url": url,
                }
            )

    response = await client.responses.create(
        model="gpt-5",
        input=[
            {
                "role": "user",
                "content": content,
            }
        ],
    )

    raw = response.output_text

    try:
        return json.loads(raw)
    except json.JSONDecodeError:

        logger.error("AI returned invalid JSON: %s", raw)

    return {
        "operation": None,
        "property_type": None,
        "city": None,
```

```

        "district": None,
        "neighborhood": None,
        "address": None,
        "area_m2": None,
        "rooms": None,
        "bathrooms": None,
        "floor": None,
        "total_floors": None,
        "building_age": None,
        "furnished": None,
        "price": None,
        "currency": None,
        "description": None,
        "owner_name": None,
        "owner_phone": None,
        "conflicts": ["تعذر تحليل البيانات"],
        "missing_fields": [],
    }

```

```

# =====
# FORMAT PROPERTY
# =====

```

```
def format_property(data):
```

```

    return f"""
    🏠 نوع العقار : {data.get("property_type") or "غير محدد"}

    📅 العملية : {data.get("operation") or "غير محدد"}

    📍 المدينة : {data.get("city") or "غير محدد"}
    📍 المنطقة : {data.get("district") or "غير محدد"}
    📍 الحي : {data.get("neighborhood") or "غير محدد"}

    📐 م2 : {data.get("area_m2") or "غير محدد"} المساحة

    🛏️ الغرف : {data.get("rooms") or "غير محدد"}

    🚿 الحمامات : {data.get("bathrooms") or "غير محدد"}

    🏢 الطابق : {data.get("floor") or "غير محدد"}

    💰 السعر : {data.get("price") or "غير محدد"} {data.get("currency") or ""}

    📝 الوصف :
    {data.get("description") or "غير موجود"}
    """

```

```
"""
```

```
# =====  
# COMMAND START  
# =====
```

```
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
```

```
    await update.message.reply_text(  
        """
```

```
        """
```

🧠 أهلاً بك في مساعد مكتب الشهباء العقاري

. أرسل لي تفاصيل العقار كنص أو صور أو الاثنين معاً

مثال:

شقة للبيع بالحمدانية

غرف 3

متر 140

الطابق الثالث

"السعر 45 ألف دولار"

. وسأستخرج المعلومات وأطلب منك تأكيدها قبل حفظها

الأوامر:

/add - إضافة عقار

/search - البحث

/list - عرض العقارات

/stats - الإحصائيات

```
"""
```

```
    )
```

```
# =====  
# ADD PROPERTY  
# =====
```

```
async def add_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
```

```
    context.user_data["adding_property"] = True
```

```
    context.user_data["searching"] = False
```

```
    context.user_data["pending_images"] = []
```

```
    await update.message.reply_text(  
        """
```

📬 رسل الآن نص العقار أو صور العقار (يمكنك إرسال عدة صور، ثم اكتب انتهيت)

```

)

# =====
# PROCESS EXTRACTED DATA (shared by text & photo flow)
# =====

async def process_extraction(update: Update, context: ContextTypes.DEFAULT_TYPE,

    await update.message.reply_text(
        "🧠 جارٍ تحليل بيانات العقار..."
    )

    data = await extract_property(text, image_urls=image_urls)

    context.user_data["pending_property"] = data

    conflicts = data.get("conflicts") or []
    missing = data.get("missing_fields") or []

    message = format_property(data)

    if conflicts:

        message += "\n\n⚠️ تعارضات:\n"

        for conflict in conflicts:
            message += f"- {conflict}\n"

    if missing:

        message += "\n\n⚠️ معلومات ناقصة:\n"

        for field in missing:
            message += f"- {field}\n"

    keyboard = [
        [
            InlineKeyboardButton(
                "✅ تأكيد وحفظ",
                callback_data="confirm_property",
            ),
            InlineKeyboardButton(
                "❌ إلغاء",
                callback_data="cancel_property",
            ),
        ]
    ]

```

```

]

await update.message.reply_text(
    message,
    reply_markup=InlineKeyboardMarkup(keyboard),
)

# =====
# HANDLE PHOTOS
# =====

async def handle_photo(update: Update, context: ContextTypes.DEFAULT_TYPE):

    if not context.user_data.get("adding_property"):

        await update.message.reply_text(
            "أولاً لإضافة عقار /add أرسل"
        )

        return

    photo = update.message.photo[-1]
    file = await context.bot.get_file(photo.file_id)

    pending_images = context.user_data.setdefault("pending_images", [])
    pending_images.append(file.file_path)

    caption = update.message.caption

    if caption:
        # Caption present: treat this as the final message, run extraction now
        await process_extraction(update, context, caption, image_urls=pending_images)
        context.user_data["pending_images"] = []
    else:
        await update.message.reply_text(
            f"📷 تم استلام الصورة ({len(pending_images)}). "
            "أرسل المزيد من الصور، أو اكتب النص/التفاصيل الآن لبدء التحليل"
        )

# =====
# HANDLE TEXT (routes between add / search / idle)
# =====

async def handle_text(update: Update, context: ContextTypes.DEFAULT_TYPE):

```

```

text = update.message.text

if context.user_data.get("searching"):
    await search_text(update, context)
    return

if context.user_data.get("adding_property"):

    pending_images = context.user_data.get("pending_images") or []

    await process_extraction(update, context, text, image_urls=pending_images

    context.user_data["pending_images"] = []
    return

    await update.message.reply_text(
        "للبحث /search أولاً لإضافة عقار، أو add /أرسل"
    )

# =====
# CONFIRM
# =====

async def confirm_property(update: Update, context: ContextTypes.DEFAULT_TYPE):

    query = update.callback_query

    await query.answer()

    data = context.user_data.get("pending_property")

    if not data:

        await query.edit_message_text(
            "❌ لا يوجد عقار بانتظار التأكيد"
        )

        return

    if data.get("conflicts"):

        await query.edit_message_text(
            "⚠️ لا يمكن الحفظ بسبب وجود تعارض في المعلومات"
        )

    return

```

```

record = {
    "status": "available",
    "operation": data.get("operation"),
    "property_type": data.get("property_type"),
    "city": data.get("city"),
    "district": data.get("district"),
    "neighborhood": data.get("neighborhood"),
    "address": data.get("address"),
    "area_m2": data.get("area_m2"),
    "rooms": data.get("rooms"),
    "bathrooms": data.get("bathrooms"),
    "floor": data.get("floor"),
    "total_floors": data.get("total_floors"),
    "building_age": data.get("building_age"),
    "furnished": data.get("furnished"),
    "price": data.get("price"),
    "currency": data.get("currency"),
    "description": data.get("description"),
    "owner_name": data.get("owner_name"),
    "owner_phone": data.get("owner_phone"),
    "source": "telegram",
    "source_url": None,
}

saved = await add_property(record)

context.user_data.pop("pending_property", None)
context.user_data["adding_property"] = False
context.user_data["pending_images"] = []

await query.edit_message_text(
    f"""
     تم حفظ العقار بنجاح.

     رقم العقار:
    {saved["property_code"]}

    وضع العقار:
     متاح
    """
)

# =====
# CANCEL
# =====

```

```

async def cancel_property(update: Update, context: ContextTypes.DEFAULT_TYPE):

    query = update.callback_query

    await query.answer()

    context.user_data.pop("pending_property", None)
    context.user_data["adding_property"] = False
    context.user_data["pending_images"] = []

    await query.edit_message_text(
        "تم إلغاء إضافة العقار ❌."
    )

# =====
# SEARCH
# =====

async def search_command(update: Update, context: ContextTypes.DEFAULT_TYPE):

    context.user_data["searching"] = True
    context.user_data["adding_property"] = False

    await update.message.reply_text(
        """
🔍 اكتب طلب البحث بشكل طبيعي .

مثال:

بدي شقة بالحمدانية أقل من 50 ألف دولار 3 غرف
"""
    )

async def perform_search(query_text):

    prompt = f"""
JSON. حوّل طلب البحث التالي إلى

.لا تخرع شروطاً غير موجودة

الحقول:

property_type
city

```

district
neighborhood
min_price
max_price
currency
rooms
min_area
max_area

طلب العميل:
{query_text}

فقط JSON أخرج
"""

```
response = await client.responses.create(
    model="gpt-5",
    input=prompt,
)

try:
    return json.loads(response.output_text)

except Exception:

    return {}

def _matches(p, criteria):

    if criteria.get("property_type"):
        if not p.get("property_type") or criteria["property_type"].lower() not in
            return False

    if criteria.get("city"):
        if not p.get("city") or criteria["city"].lower() not in p["city"].lower()
            return False

    if criteria.get("district"):
        if not p.get("district") or criteria["district"].lower() not in p["distri
            return False

    if criteria.get("rooms") is not None:
        if p.get("rooms") != criteria["rooms"]:
            return False

    if criteria.get("min_price") is not None:
```

```

        if p.get("price") is None or p["price"] < criteria["min_price"]:
            return False

    if criteria.get("max_price") is not None:
        if p.get("price") is None or p["price"] > criteria["max_price"]:
            return False

    if criteria.get("min_area") is not None:
        if p.get("area_m2") is None or p["area_m2"] < criteria["min_area"]:
            return False

    if criteria.get("max_area") is not None:
        if p.get("area_m2") is None or p["area_m2"] > criteria["max_area"]:
            return False

    return True

async def search_text(update: Update, context: ContextTypes.DEFAULT_TYPE):

    query_text = update.message.text

    criteria = await perform_search(query_text)

    properties = await load_properties()

    available = [p for p in properties if p.get("status") == "available"]

    matched = [p for p in available if _matches(p, criteria)]

    context.user_data["searching"] = False

    if not matched:

        await update.message.reply_text(
            "❌ لم أجد عقارات مطابقة لطلبك."
        )

        return

    text = f"🔍 وجدت {len(matched)} عقار:\n\n"

    for p in matched[:20]:

        text += f"""
🏠 {p.get("property_code", "-")}
🏠 {p.get("property_type") or "-"}

```

```

📍 {p.get("city") or "-"} - {p.get("district") or "-"}
📐 {p.get("area_m2") or "-"} م2
🏠 {p.get("rooms") or "-"} غرف
💰 {p.get("price") or "-"} {p.get("currency") or ""}

-----
"""

    await update.message.reply_text(text)

# =====
# LIST
# =====

async def list_properties(update: Update, context: ContextTypes.DEFAULT_TYPE):

    properties = await load_properties()

    available = [p for p in properties if p.get("status") == "available"]

    available.sort(key=lambda p: p.get("id", 0), reverse=True)

    latest = available[:20]

    if not latest:

        await update.message.reply_text(
            "لا توجد عقارات مسجلة."
        )

        return

    text = "🏠 آخر العقارات:\n\n"

    for p in latest:

        text += f"""
🆔 {p.get("property_code", "-")}
📍 {p.get("city") or "-"} - {p.get("district") or "-"}
🏠 {p.get("property_type") or "-"}
💰 {p.get("price") or "-"} {p.get("currency") or ""}
"""

    await update.message.reply_text(text)

```

```

# =====
# STATS
# =====

async def stats(update: Update, context: ContextTypes.DEFAULT_TYPE):

    properties = await load_properties()

    total = len(properties)

    available = len(
        [p for p in properties if p.get("status") == "available"]
    )

    sold = len(
        [p for p in properties if p.get("status") == "sold"]
    )

    await update.message.reply_text(
        f"""
📊 إحصائيات مكتب الشهباء
🏠 إجمالي العقارات: {total}
🟢 المتاحة: {available}
🔴 المباعة: {sold}
        """
    )

# =====
# MAIN
# =====

def main():

    application = (
        Application.builder()
        .token(BOT_TOKEN)
        .build()
    )

    application.add_handler(
        CommandHandler("start", start)
    )

```

```

application.add_handler(
    CommandHandler("add", add_command)
)

application.add_handler(
    CommandHandler("search", search_command)
)

application.add_handler(
    CommandHandler("list", list_properties)
)

application.add_handler(
    CommandHandler("stats", stats)
)

application.add_handler(
    CallbackQueryHandler(
        confirm_property,
        pattern="^confirm_property$",
    )
)

application.add_handler(
    CallbackQueryHandler(
        cancel_property,
        pattern="^cancel_property$",
    )
)

application.add_handler(
    MessageHandler(
        filters.PHOTO,
        handle_photo,
    )
)

application.add_handler(
    MessageHandler(
        filters.TEXT & ~filters.COMMAND,
        handle_text,
    )
)

async def startup(app):

    await init_storage()

```

```
application.post_init = startup
```

```
logger.info("Alshahba AI started")
```

```
application.run_polling()
```

```
if __name__ == "__main__":  
    main()
```